Online Knowledge Sharing System — Technical Design Document

Tech Stack & Architecture

This project is built using the MERN stack, which includes MongoDB, Express.js, React.js, and Node.js. The architecture follows a client-server model where the frontend communicates with backend APIs, and the backend interacts with the database.

User Module

- Each user has a unique email ID and encrypted password.
- Authentication is handled using JWT tokens.
- Users can create profiles, edit information, and manage posts.
- Role-based access can be implemented for admin and regular users.

Request Module

- Users can submit knowledge requests or questions.
- Requests are stored in the database and linked to the requesting user.
- Other users can view and respond to requests.
- Status tracking allows monitoring of resolved and pending requests.

Database Schema & Entity Design

- User collection stores profile and authentication data.
- Posts collection stores knowledge articles and answers.
- Tags array helps categorize content.
- Relationships are maintained using Object IDs.

Notification System

- Users receive alerts for replies, comments, and updates.
- Notifications are stored and can be marked as read.
- Real-time updates can be implemented using WebSockets.

System Architecture & Data Flow

- User interacts with a React component.

• Component sends API request via Axios.

• Request reaches Node.js Express route.

• JWT middleware validates authentication.

• Controller processes logic and calls service layer.

• Service interacts with MongoDB database.

• Response is returned to frontend and UI updates.

Core Entities

• User – creates posts, comments, and requests.

• KnowledgePost – contains shared information.

• Category – classifies posts.

• Comment – stores feedback or discussion.

• Notification – alerts users about activity.

Relationships

• One user can create many posts.

• One post can have many comments.

• Each post belongs to one category.

• Notifications belong to a specific user.

UI/UX Design Principles

• Clean and minimal interface.

• Responsive layout for mobile and desktop.

• Accessible color contrast and typography.

• Intuitive navigation for posting and searching knowledge.

• Consistent component styling across pages.

Project Workflow

• Version control managed using Git.

• Feature branches used for development.

• Pull requests used for code review.

• Continuous integration can be added for automated testing.

Final System Summary

The Online Knowledge Sharing System provides a scalable platform where users can share information, ask questions, collaborate, and learn from each other. The architecture is modular, maintainable, and suitable for real-world deployment with enhancements such as caching, load balancing, and real-time updates.

System Architecture Diagram